

Homework 4 :: MATH 5152

Due: Monday, February 17 at 11:59 AM

Submission instructions: Submit a single PDF file named LASTNAME-hw4.pdf. All solutions must be typed in $\text{\LaTeX}$. All coding must be done in **Python** using **NumPy**. Upload your PDF to Canvas before the deadline.

1. (Coding Problem: Eigenvalues and the Power Method)

Construct any 3×3 invertible symmetric matrix with no entry equal to 0.

Let $A = \begin{bmatrix} 6 & 3 & 1 \\ 3 & 5 & 2 \\ 1 & 2 & 4 \end{bmatrix}$.

- (a) Using **Python** and the function `numpy.linalg.eig`, compute the dominant eigenvalue $\lambda_{\max}^*$ and its corresponding eigenvector v^* .

```
1 import numpy as np
2
3 A = np.array([[6, 3, 1], [3, 5, 2], [1, 2, 4]], dtype=float)
4
5 w, V = np.linalg.eig(A)
6 idx = np.argmax(w.real)
7 lam_star = w[idx].real
8 v_star = V[:, idx].real
9 v_star = -v_star if v_star.sum() < 0 else v_star
10 v_star = v_star / np.linalg.norm(v_star)
11
12 print("dominant eigenvalue:", lam_star)
13 print("dominant eigenvector:", v_star)
```

dominant eigenvalue: 9.348493934350047

dominant eigenvector: [0.67934373 0.63657455 0.3650547]

- (b) Implement the **Power Method** in Python to compute the approximate dominant eigenvector $v^{(k)}$ and eigenvalue μ_k of the matrix.

For each stopping tolerance ϵ , iterate until

$$\frac{\|v^{(k)} - v^{(k-1)}\|_2}{\|v^{(k-1)}\|_2} < \epsilon.$$

```
1 def power_method(A, epsilon, max_iter=100000, x0=None):
2     n = A.shape[0]
3     if x0 is None:
4         x = np.ones(n)
5     else:
6         x = np.array(x0, dtype=float)
7
8     x = x / np.linalg.norm(x)
```

```

9
10     prev = x.copy()
11     k = 0
12
13     while k < max_iter:
14         y = A @ x
15         x = y / np.linalg.norm(y)
16         rel_change = np.linalg.norm(x - prev) /
17                     np.linalg.norm(prev)
18
19         if rel_change < epsilon:
20             break
21         prev = x.copy()
22         k += 1
23
24     mu = (x @ (A @ x)) / (x @ x)
25     return x, mu, k+1, rel_change
26
27 epsilon_values = [1e-3, 1e-6, 1e-9]
28
29 for epsilon in epsilon_values:
30     v_k, mu_k, iters, rel_change = power_method(A, epsilon)
31
32     eig_error = abs(mu_k - lam_star)
33     vec_error = np.linalg.norm(v_k - v_star) /
34                 np.linalg.norm(v_star)
35
36     print("epsilon:", epsilon)
37     print("number of iterations:", iters)
38     print("eigenvalue error:", eig_error)
39     print("eigenvector error:", vec_error)
40     print("final relative change:", rel_change)
41
42     print("\n")

```

```

epsilon: 0.001
number of iterations: 7
eigenvalue error: 7.105817143582271e-07
eigenvector error: 0.0003556325469840886
final relative change: 0.0005356872732264721

```

```

epsilon: 1e-06
number of iterations: 14
eigenvalue error: 1.8385293287792592e-12
eigenvector error: 5.726590846818378e-07
final relative change: 8.625343810033565e-07

```

```

epsilon: 1e-09

```

```

number of iterations: 22
eigenvalue error: 5.329070518200751e-15
eigenvector error: 3.67943953605171e-10
final relative change: 5.541944425457898e-10

```

(c) For each value of ϵ , record the results in the table below.

As shown in the output above in part (b), we have:

ϵ	Iterations	$ \mu_k - \lambda_{\max}^* $	$\frac{\ v^{(k)} - v^*\ _2}{\ v^*\ _2}$	$\frac{\ v^{(k)} - v^{(k-1)}\ _2}{\ v^{(k-1)}\ _2}$
10^{-3}	7	7.1058×10^{-7}	3.5563×10^{-4}	5.3569×10^{-4}
10^{-6}	14	1.8385×10^{-12}	5.7266×10^{-7}	8.6253×10^{-7}
10^{-9}	22	5.3291×10^{-15}	3.6794×10^{-10}	5.5419×10^{-10}

Remark: In practice, the exact eigenvector v^* is unknown. Therefore, the stopping criterion based on successive iterates should be used in implementations.

2. (Coding Problem: PageRank)

Using Python and NumPy, construct a connected network graph with 5 nodes (web pages).

(a) Represent the network using an adjacency or transition matrix.

Let us define 5 pages, where the first page is connected to all of the other pages, the second page is connected only to the third and fourth pages, the third page is connected to the first, fourth, and fifth pages, and the fifth page is connected to the first and fourth pages. So the adjacency matrix is as follows:

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

(b) Implement the PageRank iteration discussed in lecture.

```

1 A = np.array([
2     [0,1,1,1,1], # page 1
3     [0,0,1,1,0], # page 2
4     [1,0,0,1,1], # page 3
5     [1,0,1,0,0], # page 4
6     [1,0,0,1,0]  # page 5
7 ], dtype=float)
8
9 row_sums = A.sum(axis=1, keepdims=True)
10 P = A / row_sums
11
12 alpha = 0.85
13 n = P.shape[0]
14
15 # handle dangling rows

```

```

16 P2 = P.copy()
17 row_sums = P.sum(axis=1, keepdims=True)
18 dangling = (row_sums.flatten() == 0)
19
20 if np.any(dangling):
21     P2[dangling, :] = 1.0 / n
22
23 # Google matrix
24 G = alpha * P2 + (1 - alpha) * (np.ones((n, n)) / n)
25
26 # power iteration
27 r = np.ones(n) / n # uniform start
28
29 for _ in range(200):
30     r = r @ G
31     r = r / r.sum()
32
33 print("pagerank:", r.round(4))
34 print("rank order (1 highest):", (np.argsort(-r)+1))

```

```

pagerank: [0.27  0.0874 0.2333 0.2558 0.1535]
rank order (1 highest): [1 4 3 5 2]

```

(c) Identify the highest-ranked web page.

Clearly describe your network and briefly interpret your results.

The network is described above before the definition of the adjacency matrix A . As shown from the output above, the highest-ranked page is page 1, followed by 4, 3, 5, and 2. This makes intuitive sense: page 1 is the highest because it gets links from highly-ranked pages like 3, 4, and 5. Page 2, on the other hand, is the lowest ranked because it's only being pointed to by page 1.